Experiment 8: Alpha-Beta Pruning

Aim:

To implement Alpha-Beta Pruning to reduce the number of nodes evaluated in the minimax game tree while preserving optimality.

Algorithm:

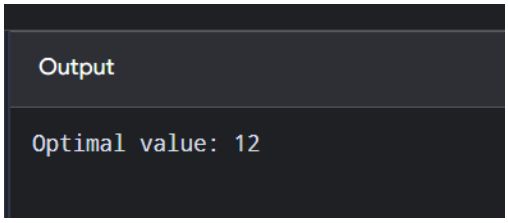
1. Maintain alpha (best max so far) and beta (best min so far).
2. Prune a branch when beta <= alpha (no need to explore further).
3. Maximizer updates alpha; Minimizer updates beta.
4. Results are identical to minimax but with fewer evaluations

Code:

```
def alphabeta(depth, nodeIndex, maximizing, values, alpha, beta, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if maximizing:
        best = float('-inf')
        for i in range(2):
            val = alphabeta(depth+1, nodeIndex*2+i, False, values, alpha, beta,
maxDepth)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha: break
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = alphabeta(depth+1, nodeIndex*2+i, True, values, alpha, beta,
maxDepth)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha: break
        return best

values = [3, 5, 2, 9, 12, 5, 23, 23]
print("Optimal value:", alphabeta(0, 0, True, values, float('-inf'), float('inf'),
3))
```

Sample Output:



Result:

Alpha-Beta Pruning produced the same optimal value as Minimax (12) while pruning unnecessary branches, improving efficiency.